

PARA REVIEWER

About Para:

- Para Mobile is an AI-assisted multimodal transit navigation app for the Philippines that helps commuters plan trips across jeepney, bus, tricycle, train networks, and more. It analyzes available routes and recommends options based on travel time, fare, number of transfers, or a balanced combination of these factors. The app also features an interactive map, personalized commute tracking, and a gamified reward system with blockchain-backed achievements.

Problem, Solution, and Overview:

What

- Public transport information in the Philippines is scattered and confusing - informal, community-known jeepney, tricycle, and UV Express routes are rarely documented anywhere mainstream mapping tools can see.

Why

- Commuters lose time, money, and confidence every day trying to piece together routes from word-of-mouth.

Why Now

- The pain is growing as congestion rises, while mobile adoption and available transport data have matured enough to make a real-world solution deployable today.

The Solution

- Para recommends the fastest, cheapest, and least-transfer routes across jeepney, bus, and tricycle networks using AI-assisted insights. It combines fare awareness, transfer simplicity, and safety into a single decision tool for commuters.

Tech Stack (Mobile Application):

Frontend:

- React Native + Expo + Typescript - builds the mobile application itself
- Nativewind - Tailwind for React Native (CSS)
- Zustand + AsyncStorage - saves app state so it survives app restarts (profile, chatbot state, and session data)

Map Layer:

- MapLibre - renders the interactive map with a custom vector style / map style
- Azure Blob Storage - where the map is stored and includes separate light/dark variants. Map style can be updated without an app store update.
- If custom style fails to load, simpler back up style is the fallback

Backend - Supabase:

- Postgres DB - for geospatial queries

- Routes are split into 4 tables: Jeepney, Tricycle, Bus, UV Express paired with Stops Table
- App tries Supabase first, and only falls back to a 1-hour AsyncStorage cache if the live fetch fails

Route Computation:

- Turf.js + custom route engine - calculates and ranks routes (fastest/cheapest/easiest) directly on the phone

Jeepie - AI Companion:

- Powered by Groq (Llama 3.1 8B)
- Uses two-stage pipeline:
 - Guardrail first to classify whether a message is safe or in scope.
 - Strict System Prompt limiting to PARA and general commuting topics
- Local Dataset (Rule-based, JSON) - handle common queries, general questions, and FAQs before they reach Groq API.

Category	Stack
Mobile Framework	React Native, Expo, Expo Router
Map Engine	MapLibre React Native
Backend	Supabase (Postgres, Auth, Realtime)
Geospatial Tools	Turf.js, custom route search engine
State Management	Zustand + AsyncStorage
Styling	Nativewind + StyleSheet
Device APIs	Expo Location, Expo Notifications, Expo Haptics
Language	TypeScript 5

Tech Stack (Admin Side):

- Next.js + React + Typescript - Builds the Web System
- Supabase - Same DB as the mobile app so both sides stay in sync
- Leaflet + Leaflet.heat - render maps specifically for traffic maps and heatmaps respectively.
- Recharts - powers charts and stats
- Tailwind CSS - for Styling

Key Features (Mobile App):

- Map and Multi-Modal Route Engine - Shows fastest, cheapest, and least transfer options with fare breakdowns
- Jeepie AI - includes command mode similar to Tarsi where it can plot you to your destination with just a simple instruction on the chatbot. Also answers general questions, commuting-related questions, FAQs, and more.

- History - for past searches and recently viewed routes
- Saved Routes - bookmarked routes with fair details for repeat trips
- Profile - includes trip stats, distance travelled, points, and global ranking system.
- Gamification - leaderboards and unlockable badges; additional blockchain for future improvements

Key Features (Admin Side):

- Route management - for jeepneys, tricycles, buses, UV Express, with per-mode fare settings and GPX Route Import
- Live Tab - Live traffic check map and a safe mode map, with route recommendations
- Alerts - manual broadcast alerts (can be used for Libreng Sakay notices, and for Advertisements); Pushed to commuters as notifications
- Users logs and analytics on fleet distributions and top commuters. Includes heatmaps to know what location to prioritize (for LGUs)

Target Users & Beneficiaries

- For Daily Commuters
- Tourists and Travelers
- LGUs and Transport Cooperatives
- First-time and Late-night Commuters

Impact & Feasibility

- Saves time and commuting costs
- Reduces confusion through AI route insights
- Enhances user experience with gamified commuting
- Encourages consistent and smarter route choices
- Build on existing map and transport data
- Scalable across cities in the Philippines

Proposed Future Improvements

- **Live Transit Tracking**
 - **Libreng Sakay Vehicle Tracking** - real-time tracking of vehicles running “Libreng Sakay” programs, so commuters see exactly where a free-ride vehicle is and when it reaches their stop.
 - **Live Train Tracking** - extends Para beyond road-based transport into rail, showing live LRT/MRT positions or arrival estimates.
- **Night Route** - Mainly used for safety, prioritizing routes that are well-lit, close to open establishments, or transit terminals, and commonly travelled, over routes that are dark, isolated, or unfamiliar. For late nights or early morning commute.
- **Crowd-Sources Traffic and Prediction** - Almost similar to Google Maps or Waze, Para checks for congestion based on how many active users are clustered in an area, treating user density as a live traffic signal. Over time, historical data could feed a prediction model that forecasts congestion before a trip even starts.

- **Blockchain Gamification and Sponsor-Backed Rewards** - Connecting Achievements or badges as Points system similar to PasaHero, or Bing Rewards. Can unlock vouchers or perks funded by sponsors.
- **Suggestion or Add Missing Route based on Users / for broken road** - Add a way for users to suggest or add missing routes or broken road / routes; approval would show on the admin side and will need future or secondary verification before accepting.

Process:

Getting into Para Admin:

An admin goes to the dashboard and logs in. Before any page loads, a checkpoint (middleware) checks "is this person actually logged in?" If not, they get bounced to the login page. Once in, they land on the dashboard with Routes, Live, Alerts, Gamification, Users, and User Logs.

Admin makes a change: it's saved to the shared database

Whatever an admin does - updates a fare, imports a new route from a GPX file, sends a broadcast alert, creates a new achievement badge, bans a user - that action writes directly to a table in the shared Supabase database. Nothing lives only on the admin side; it all lands in the same central data store the mobile app also reads from.

Para Mobile reads that same data:

When a commuter opens the app, it pulls routes, fares, and active alerts from that same database. If there's no signal or Supabase is slow, it falls back to a cached copy saved on the phone from the last successful load, so the app doesn't just break.

Alerts happen live, not on a delay:

This part's neat: when an admin sends an alert, it's written to a broadcasts table, and the mobile app is subscribed to that table in real time. So the moment an admin hits send, it shows up on commuters' phones immediately - no polling, no refresh needed.

Commuter uses the app, that becomes data too:

When a user searches for a route, the app does the actual route math (fastest/cheapest/least-transfer) right on the phone. When they complete a trip, that trip gets logged back to the database as a trip record - with a start point and end point.

That trip data feeds Para Admin's traffic and heatmap features:

This is the loop closing: those logged trips are what power the heatmap and the "traffic" view on the Live tab. Admin isn't looking at fake demo data - it's literally plotting where real commuters have been starting and ending their trips, and using how many trips pass near a route to score that route as moderate or heavy traffic.

Jeepie (the chatbot):

When a user asks Jeepie something, it first checks a local list of common questions on the phone. If nothing matches, it sends the question out to an AI model

(Groq/Llama) - but first through a quick safety check to make sure the question is on-topic, then the actual model answers.

Q & A - PREPARATION

How does Para actually make money / how do the developers sustain it

- Paid B2G or B2B Admin Side Subscriptions for LGUs and transport cooperatives.
- Data Licensing: License Anonymized, we can earn money on government transit-planning agencies like LTFRB or DOTr who don't currently have the visibility into informal transport
- Turn the achievement system into a lightweight rewards marketplace where local businesses fund vouchers in exchange for engagement data and in-app visibility.

How can the Philippines / Judges / Panel see real value from this, beyond the app?

- **Better transit planning data:** LGUs get visibility, via Admin Side or Heatmaps, to know which routes carry the most riders, supporting decisions on where to prioritize road improvement, Libreng Sakay vehicles or added fleet capacity.
- **Commuter Safety and Time Savings:** Safe Mode and better traffic prediction reduce commute risk and wasted time for large numbers of daily riders, a broad public-safety and productivity gain even where it's not directly monetized

Why is this technically credible, not just an idea?

- **Already a Working Product:** Already has a functioning mobile client, live map layer, a backend, and a customer route search engine.
- **Real Data Pipeline:** A GPX Importer Pipeline normalizes messy, real-world route geometry and stops data before it reaches the app.

How does Para differ from Google Maps or Waze?

- The comparison holds for the crowd-sourced traffic concept, but Para's core differentiator is coverage of informal transit like jeepney, tricycle, and UV Express routes that are community-known but rarely digitized elsewhere, plus fare-aware routing, an admin operations layer, and gamified engagement that typical map apps don't offer.
- In simpler terms, Para is built specifically for Public Vehicles and covers Public Routes / Routes that these vehicles go onto. It also has the Gamified Feature which gives better User Experience

Questions we need to be ready for:

- **Cold Start:** Crowd-sourced traffic needs enough active users per area to be reliable; early on, estimates may need seeding from public data until adoption grows in each pilot city. Similar to the safe mode as well.

- **Ad / Notification:** Mixing sponsored notifications with safety alerts notification.

Definition of Terms:

Term	Plain-Language Meaning
Frontend	The part of the app people actually see and tap on - the screens, buttons, and map.
Backend	The behind-the-scenes system that stores data and does the heavy lifting the app can't do on its own, like saving routes or checking logins.
Database	Where all the app's information is stored and organized - users, routes, fares, trips - like a giant, structured spreadsheet.
API	A defined way for one piece of software to ask another piece of software for information or to do something - like a waiter taking an order to the kitchen and bringing back the food.
React Native	The toolkit used to build the Para mobile app so it works on both Android and iPhone from one codebase, instead of building two separate apps.
Expo	A set of tools built on top of React Native that makes building, testing, and publishing the app faster and easier.
Expo Router	The part of the app that decides which screen you see and lets you move between screens (home, login, achievements, etc.), based on how the app's files are organized.
TypeScript	A stricter version of the JavaScript programming language that catches certain coding mistakes before the app even runs.
MapLibre	The engine that draws and renders the interactive map inside the Para app.
Leaflet	A simpler mapping tool (different from MapLibre) used specifically on the Admin Dashboard website to draw heatmaps and traffic maps.

Supabase	The service that hosts Para's database, handles user logins, and pushes live updates (like alerts) to the app - it's the backend.
PostGIS	An add-on to the database that lets it understand and calculate locations, distances, and map areas.
Next.js	The toolkit used to build the Para Admin Dashboard as a website.
Realtime / Realtime updates	When something changes in the database (like a new alert), it's pushed to the app instantly, without the user needing to refresh.
Local-first / Caching	The app saves a copy of route data on the phone itself, so it still works (a little) even with a weak or no internet connection.
CRUD	Short for Create, Read, Update, Delete - the four basic actions admins can do to manage data, like adding a new badge or removing an outdated route.
GPX file / GPX import	A standard file format that stores a path of GPS coordinates; Para Admin can import one of these files to quickly add a new transit route to the map.
Heatmap	A map that uses color (e.g. more red = more activity) to show where the most commuter trips are starting or ending.
AI chat assistant / LLM	"Jeepie," Para's chatbot, is powered by an AI language model (Llama 3.1, run through a service called Groq) that reads a user's question and writes a helpful reply.
Guardrail (AI)	A safety check that reviews a message before the AI assistant responds, to keep replies on-topic and appropriate.
State management	How the app keeps track of information that can change while it's running, like which screen you're on or your saved routes.
Middleware	A checkpoint that runs before a page loads - in Para Admin, it checks whether someone is logged in before letting them see the dashboard.

Feature-Level Weaknesses

Nothing tracks vehicles directly - everything is inferred from commuters

There's no driver-side app or GPS hardware on jeepneys/tricycles. So "live vehicle tracking," "Libreng Sakay tracking," and "live train tracking" all need a real data source that doesn't exist yet. Right now, Para only knows what commuters report through their own phones - it can't see a vehicle unless a rider using the app happens to be on it. That's a real gap between the pitch and what's technically possible without new partnerships (LTFRB, train operators, or the jeepney/tricycle operators themselves).

The flagship features need scale to even work

Crowd-sourced traffic, the heatmap, and true "avoid crowded routes" Safe Mode all depend on having enough active users in an area. In a new city with few users, these features are quiet or empty - which is exactly when a new user would judge the app and decide whether to keep it. The features that are supposed to sell the app don't work until the app already has users.

Data goes stale unless someone manually keeps updating it

Routes, fares, and terminals aren't self-updating - an admin has to manually edit them or re-import a GPX file. Jeepney routes and fares change often and informally (a driver reroutes, fuel prices push a fare change). Without a steady stream of community-reported corrections or a dedicated ops person per city, the data slowly drifts out of date, which undermines the entire "reliable" pitch.

Jeepie depends on a third-party AI provider

The chatbot runs on Groq's hosted model. If Groq changes pricing, has downtime, or rate-limits, that feature degrades or costs money at scale. There's also a real risk worth naming out loud: if the AI generates an answer instead of just retrieving verified route/fare data, it could confidently tell someone the wrong fare or route - which is worse than giving no answer at all in a transit app.

Expanding to a new city means starting from zero

There's no shortcut - every new city needs its own route mapping, fare data, and terminal data built up from scratch, plus enough new users to make the crowd-sourced features useful again. "Scalable across Philippine cities" is true in principle, but each expansion is a real, manual undertaking, not a toggle switch.

Business-Level Weaknesses

The paying customer isn't the user generating the value

Commuters use the app for free and generate the data; LGUs and cooperatives are the ones who'd actually pay. That's a classic two-sided marketplace problem - you need real commuter adoption before you have anything valuable to sell to the government, but there's no real revenue while you're building that adoption. It's a funding gap the team will need to bridge somehow (grants, pilot funding, etc.) before subscription revenue kicks in.

Near-term revenue rests on two legs, and they're the same type of buyer

The ad/sponsorship pillar is explicitly labeled "future scaling," so realistically the near-term business is B2G/B2B subscriptions plus onboarding fees - both from the same buyer type (LGUs/cooperatives). Government and cooperative sales cycles tend to be slow and budget-cycle-dependent, which is a shaky foundation if it's the only near-term revenue.

Competing with a free incumbent that people already trust

Google Maps is free, already installed, and already trusted. Getting someone to download and regularly open a second, dedicated transit app is a real behavior-change hurdle - worth having a clear answer for why commuters would switch, beyond "it covers jeepneys."

Data privacy exposure hasn't been addressed yet

Collecting trip origin/destination data tied to user accounts is exactly the kind of thing the Philippine Data Privacy Act cares about. If the plan is to pitch this to LGUs, they'll likely ask about data handling and consent before signing anything - and that story isn't built out yet.

Possible pushback from the transport sector itself

If LGUs start using Para's data to prioritize routes or allocate Libreng Sakay vehicles, existing jeepney/tricycle operators or driver associations might see that as being surveilled or sidelined rather than helped. Getting operator buy-in, not just LGU buy-in, matters for this to work long-term.

Sponsor rewards need proof they actually work

Getting a sponsor to fund vouchers requires showing them it drove real foot traffic or engagement - that kind of attribution/reporting tooling doesn't appear to exist yet, and "trust us, users are engaged" isn't usually enough to close a sponsorship deal.

Blockchain Integration in Para Transport

1. Overview

Para Transport integrates a blockchain-based reward and credential system built on the Polygon Amoy testnet.

2. Problem Statement

Traditional mobile gamification systems — such as points, badges, and reward vouchers — are stored entirely in centralized databases controlled by the application developer. This introduces several limitations:

- Lack of verifiability: Third parties cannot independently verify whether a user legitimately earned a badge or achievement.
- Centralization risk: Badges and rewards can be modified, deleted, or lost if the application's database is compromised or the service shuts down.
- No user ownership: Users do not truly own their achievements — the developer holds all records and can revoke them at any time.
- Limited interoperability: Rewards earned in one application cannot be recognized or utilized across other platforms.

3. Objectives

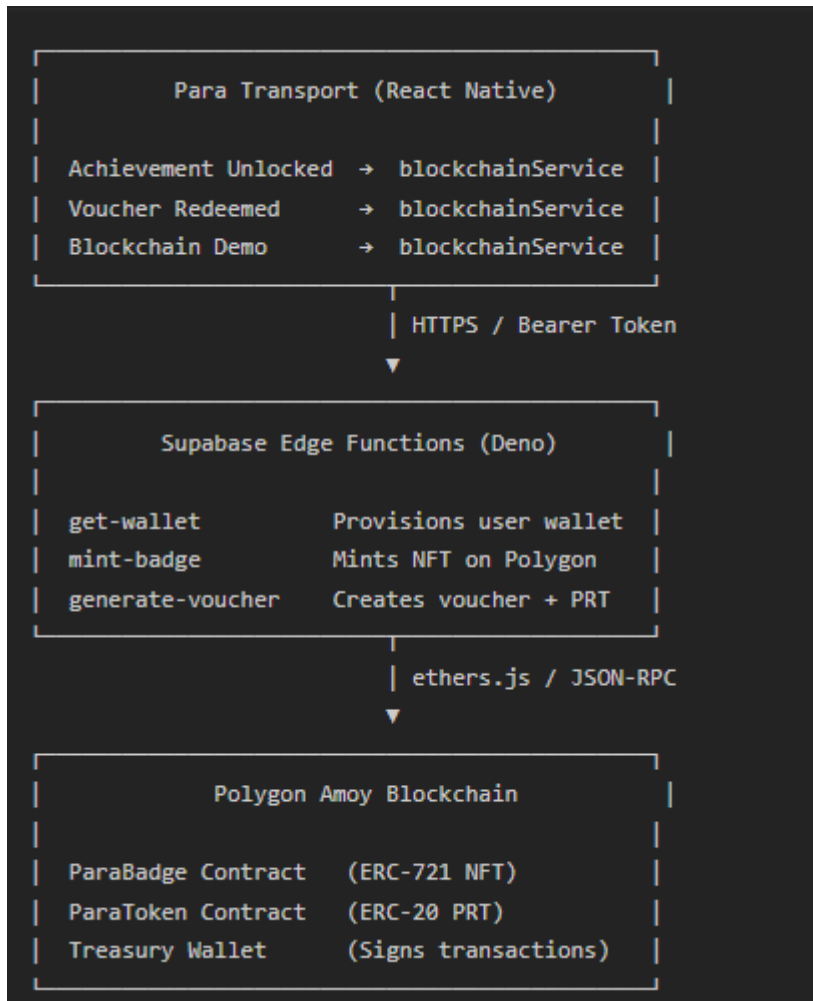
1. Convert achievement badges into non-fungible tokens (NFTs) to guarantee immutability and user ownership.
2. Record points redemption (voucher generation) on-chain to ensure transparent and auditable reward transactions.
3. Automate custodial wallet provisioning so that users can participate in the blockchain ecosystem without requiring prior knowledge of cryptocurrency or Web3.

4. Tech Stack

Component	Technology
Blockchain Network	Polygon Amoy (Testnet)
Blockchain Library	ethers.js v5

Why Polygon? Polygon was selected over Ethereum mainnet due to its significantly lower transaction fees (gas costs approximately ₱0.001–₱0.05 per transaction), faster block confirmation times (~2 seconds), and compatibility with the Ethereum Virtual Machine (EVM).

5. Architecture



Process:

User registers / logs in to Para Transport



`get-wallet` Edge Function is invoked



System checks if a wallet address exists in the users table
If none exists: new Polygon wallet is generated and stored



`wallet_address` is saved to Supabase (`users.wallet_address`)



All future NFTs and PRT tokens are deposited to this address

How to find a user's wallet address:

Step 1 — Go to Supabase SQL Editor and run:

```
SELECT username, wallet_address FROM users WHERE wallet_address IS NOT NULL;
```

Step 2 — Copy the `wallet_address` value (looks like `0x026945aa...`)

Step 3 — Go to: amoy.polygonscan.com/address/ + paste the address